Assignment 1 — Verse

> A mobile-first daily-quotes website. Your first real build — focus on clean structure and solid fundamentals over perfection.

The project

You're building **Verse**, a small daily-quotes site. The goal isn't just to make it look right — it's to build it the right way: semantic HTML, organised SCSS, and clean JavaScript.

The design is provided in Figma. All colours, spacing, font sizes, and visual details live there — pull values from the file, don't guess.

It's mobile-first. Design and write CSS for small screens first, then scale up with `min-width` media queries. Not the other way around.

What to build

Match the Figma design for each of these five sections:

1. Navbar

Brand logo, navigation links, a CTA button, and a **working hamburger menu on mobile** that opens and closes the nav.

2. Hero

A heading, supporting text, action buttons, and a featured quote displayed prominently.

3. Browse by mood

A responsive grid of category cards. The grid should reflow cleanly across screen sizes.

4. Fresh quotes

A grid of quote cards **loaded from an API** (see The API below). Handle all three states — loading, loaded, and error. GitHub Link for API Docs.

5. Footer

Brand, a few navigation links, and a copyright line.

Expectations

These aren't optional extras — they're the point of the assignment.

Semantic HTML

Use the right element for the right job. A heading is an `<h1>`–`<h6>`, not a `<div>` with a big font. Navigation is a `<nav>`. The hamburger trigger is a `<button>`, not a `<div>`. Quotes go in a `<figure>` with a `<figcaption>` for the author.

Semantic HTML isn't just style — it's what makes pages accessible to screen readers and meaningful to search engines. If you find yourself writing `<div class="button">`, stop and use `<button>`.

A few elements to know before you start: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`, `<figure>`, `<figcaption>`, `<button>`.

Reset CSS

Browsers have their own default styles — margins on headings, padding on lists, inconsistent box model behaviour. A reset removes or normalises these so you're starting from a predictable baseline.

Write your own small reset at the top of your styles. At minimum it should cover:

- `box-sizing: border-box` on everything — so padding doesn't add to an element's width
- Remove default `margin` and `padding` from elements that have them
- `img { display: block; max-width: 100%; }` — so images never overflow their container
- A sensible default `line-height` on `body`

Don't import one from a CDN. Writing it yourself means you understand what it does.

SCSS — 7-1 structure and BEM

Organise your SCSS into the 7-1 folder structure (abstracts, base, components, layout, pages, themes, vendors) and name your classes using BEM (`block__element--modifier`). Both are enforced by the linter — it will tell you when something's wrong.

Use `@use` and `@forward` to connect your files. The old `@import` is deprecated and will eventually be removed from Sass.

Mobile-first

Write your base styles for mobile, then use `min-width` media queries to adapt for larger screens. Not `max-width`. Starting from mobile forces you to prioritise content and keep layouts simple.

Git

Commit as you go — one commit per logical chunk of work, not one giant commit at the end. Your commit history should tell the story of how you built it.

Rules

What you can use

- **HTML** — write it yourself, semantic elements only
- **SCSS** — 7-1 structure, BEM naming, `@use` / `@forward`
- **Vanilla JavaScript** — for the hamburger menu and loading quotes
- **Git** — commit as you build

What you cannot use

- No CSS frameworks, resets, or libraries from a CDN
- No JS frameworks or libraries — no React, Vue, jQuery, Bootstrap, Tailwind, etc.
- No `@import` in SCSS

The API

For the **Fresh quotes** section, use DummyJSON Quotes. It's free, no API key needed, works straight from the browser.

GET https://dummyjson.com/quotes/random/6 → 6 random quotes

GET https://dummyjson.com/quotes/random → a single random quote

Each quote object looks like this:


```
{
  "id": 1,
  "quote": "Life isn't about getting and having, it's about giving and being.",
  "author": "Kevin Kruse"
```

}

Handle these three states

State	What to show
Loading	Skeleton / placeholder cards while the request is running
Loaded	The quotes — text and author
Error	A short message and a "Try again" button

> **Tip:** Add `?delay=1000` to the URL to slow the response by a second so you can actually see your loading state while building it.
> `https://dummyjson.com/quotes/random/6?delay=1000`

Resources

Don't read all of these upfront — reach for the relevant one when you're stuck.

HTML

- Learn HTML — web.dev
- HTML elements reference — MDN

CSS & SCSS

- Learn CSS — web.dev
- Sass official docs
- The 7-1 pattern — Sass Guidelines
- Flexbox guide — CSS-Tricks
- Grid guide — CSS-Tricks

BEM

- Introduction — getbem.com
- BEM 101 — CSS-Tricks

JavaScript

- Modern JavaScript Tutorial — javascript.info
- Using the Fetch API — MDN
- Async / Await — javascript.info

Git

- Git tutorials — Atlassian